

SE & Cloud Computing course

02 - Quality Assurance and Testing in SE, QA for ML, ML for QA

Robert Feldt

Division of Software Engineering , Dept of Computer Science and Engineering
Chalmers University of Technology

robert.feldt@chalmers.se



Verification and Validation

- **Verification**

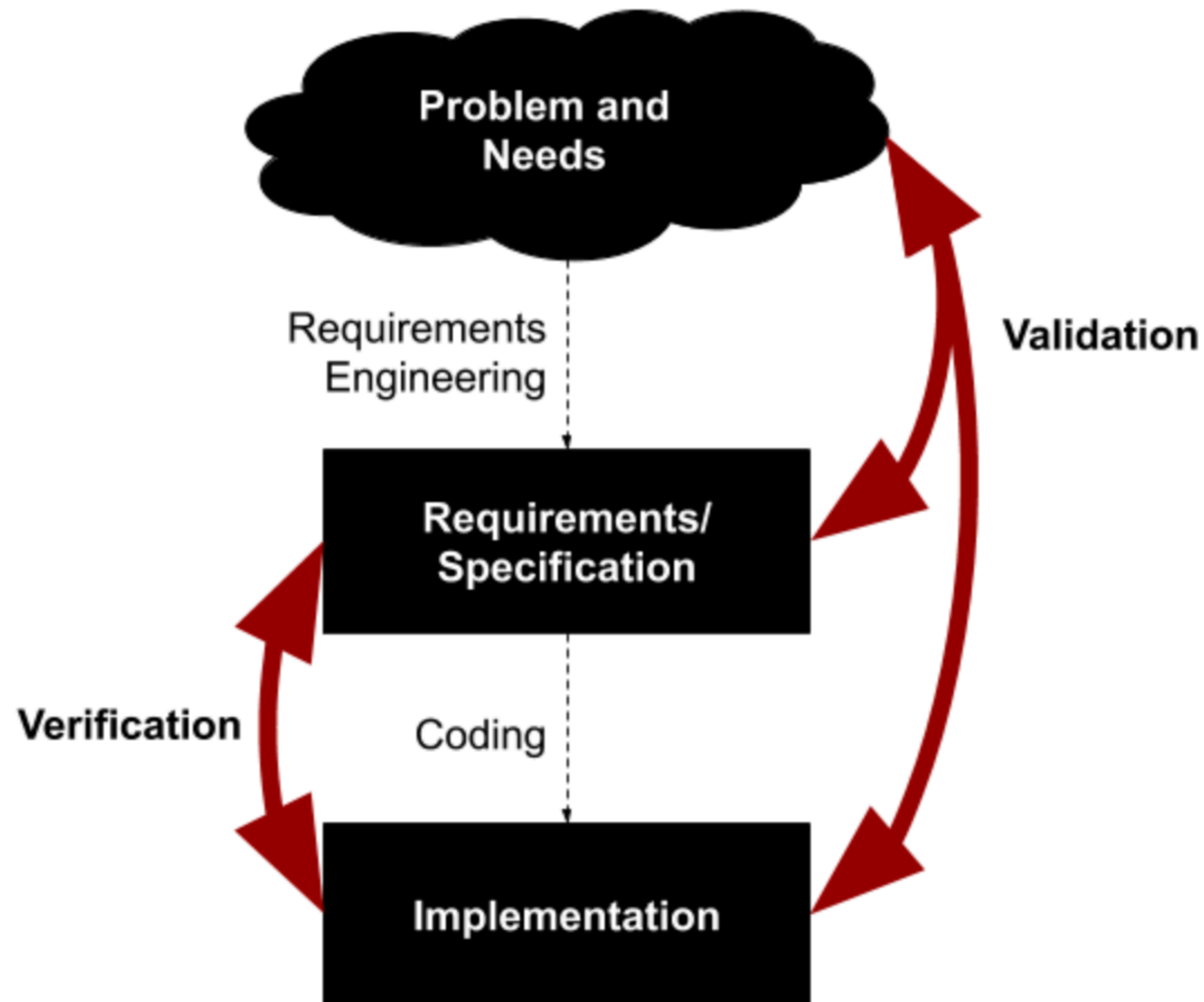
- **Checking of whether we are building the product right**

- **Validation**

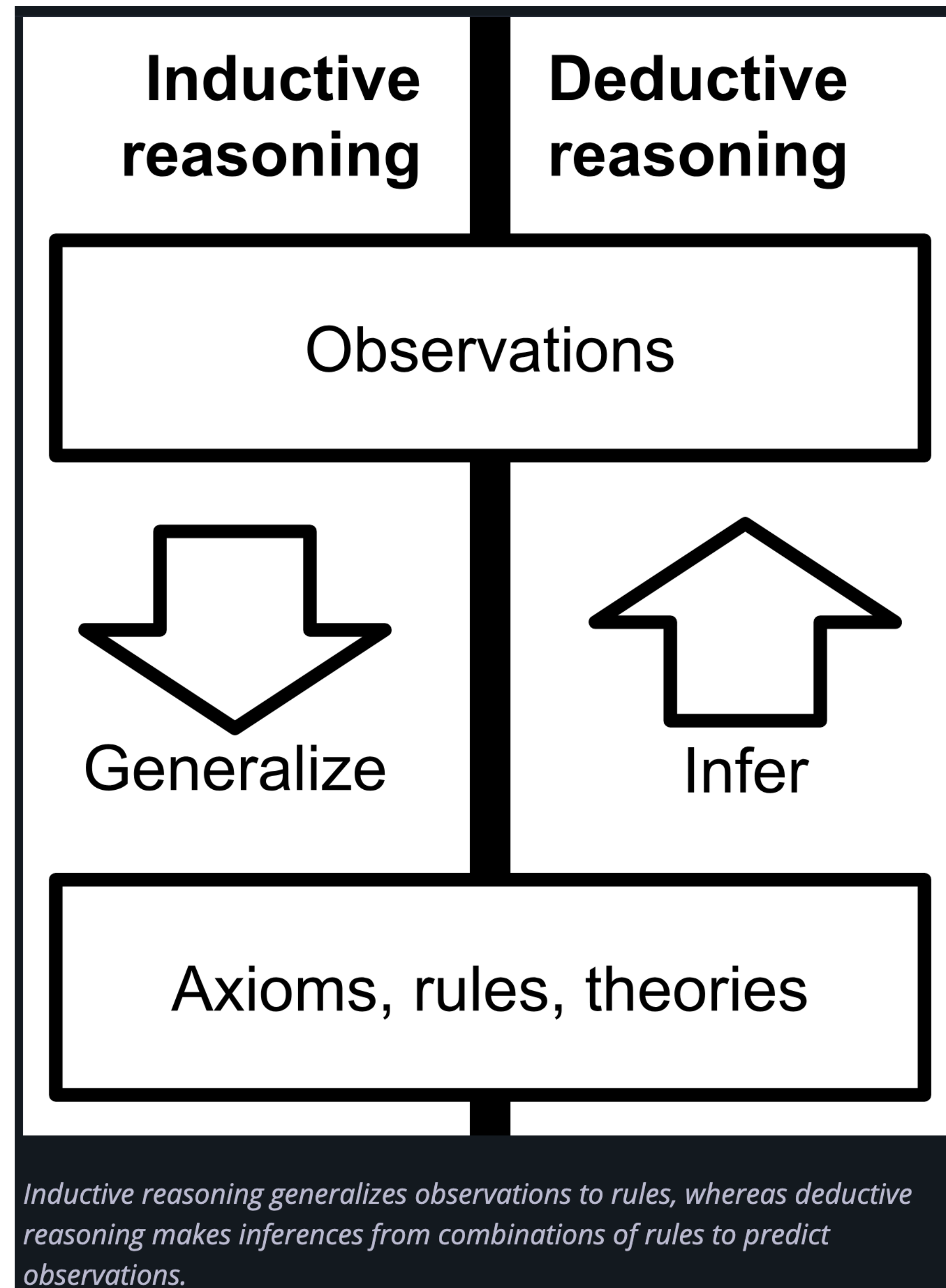
- **Checking whether we are building the right product**

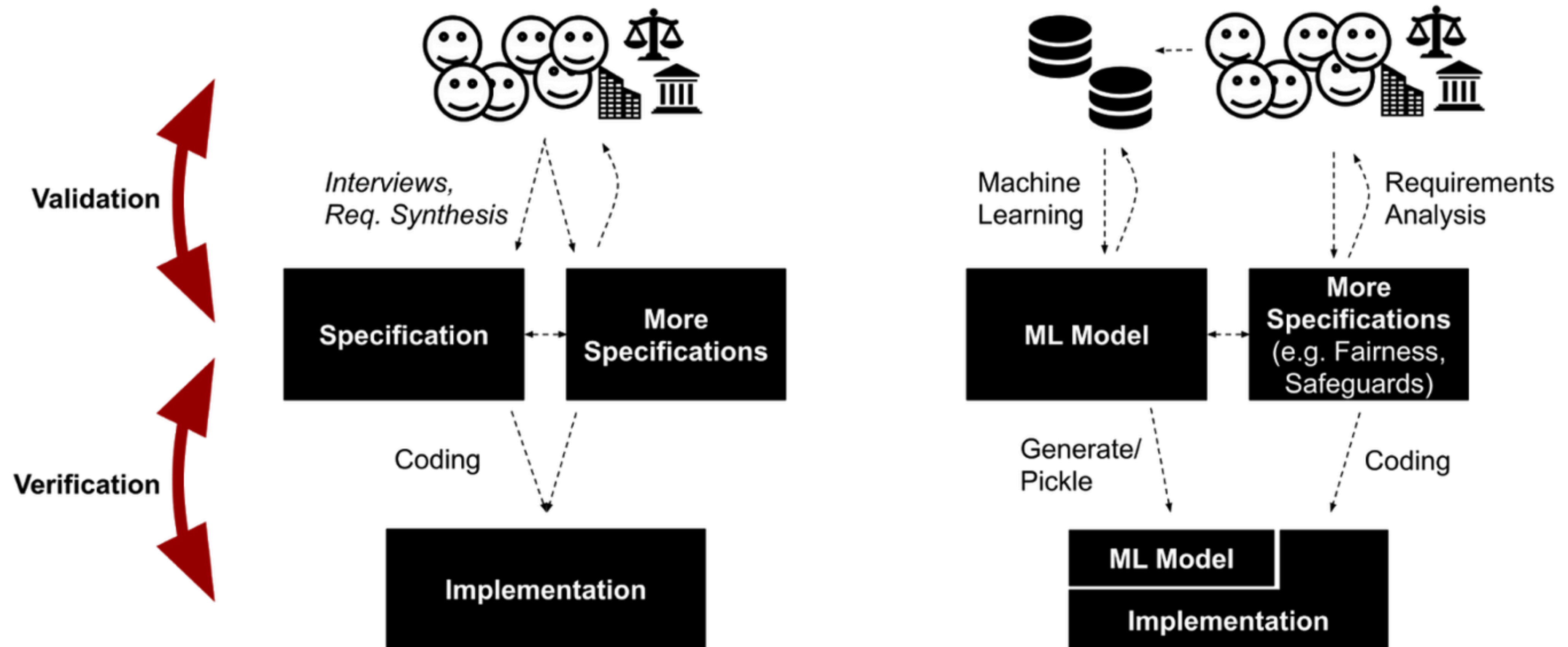
- Example

- **Hacking Wood:** If we hack someone else's wood very fast (pieces of wood/minute) then we are highly efficient (build the product right), but not effective (build the right product)
- **Developing Software:** If we develop requirements with a high velocity and introduce few defects on the way we are highly efficient (build the product right). If the software does not fulfill the needs of the customer though, we are not effective (build the right product)



Verification relates an implementation to a specification. Validation compares the specification (or the corresponding implementation) to the user's needs.





Machine Learning similar to Requirements Eng in SE: Both do inductive “reasoning” from data
ML testing thus more about validation than verification: Ensuring model fulfils real needs
Evaluate how well a model fits the problem, not whether the model meets a specification

Three main groups of techniques/ methods for verification & validation

- **Static techniques:** Do not require an executable software artefact (e.g. inspections, reviews, walkthroughs, source code analysis, linting, ...)
- **Dynamic techniques:** Require an executable software artefact (software testing in many different forms)
- **Process evaluation techniques:** analyze how the software was produced rather than the software artifacts, for example, assuring that requirements are documented and known bugs are tracked.



Why QA & testing?

- It costs a lot if things go wrong. (Economics)
- Or people die. (Safety, Robustness, Reliability)
- We will be embarrassed if SW fails, might lose customers. (PR, Marketing, Market)
- The customers/government forces us to test. (Certification)
- Most used technique in industry for ensuring SW quality. (Others use it. Will always be there.)
- You can get a job. (Traditionally overlooked)
- We are engineers, we take pride in our work (Ethics)

Complexity!

- 
- Fundamental problem is that software itself AND surrounding world AND our requirements are all **complex and interact!**
 - Full predictability is hard/impossible!

Three main groups of techniques/ methods for verification & validation

- **Static techniques:** Do not require an executable software artefact (e.g. inspections, reviews, walkthroughs, source code analysis, linting, ...)
- **Dynamic techniques:** Require an executable software artefact (software testing in many different forms)
- **Process evaluation techniques:** analyze how the software was produced rather than the software artifacts, for example, assuring that requirements are documented and known bugs are tracked.



Manual Static Analysis

- **Inspections** = formal, synchronized review of artefact X (requirements, design, code, or tests, typically) by a group of prepared engineers
 - Goal is defect finding
 - Still used as part of certification and in more safety-critical dev
- **Walkthroughs** = less formal and less prepared
- **Code review** = single peer that reviews your code (this is the one that “survived”)
 - Heavily used by many companies, notably Google but many others

Code review practices at Google

Finding 3. *The Google code review process is aligned with the convergent practice of being lightweight and flexible. In contrast to other studied systems, however, ownership and readability are explicit and play a key role. The review tool includes reviewer recommendation and code analysis results.*

Finding 4. *Code review at Google has converged to a process with markedly quicker reviews and smaller changes, compared to the other projects previously investigated. Moreover, one reviewer is often deemed as sufficient, compared to two in the other projects.*

Finding 5. *Despite years of refinement, code review at Google still faces breakdowns. These are mostly linked to the complexity of the interactions that occur around the reviews. Yet, code review is strongly considered a valuable process by developers, who spend around 3 hours a week reviewing.*

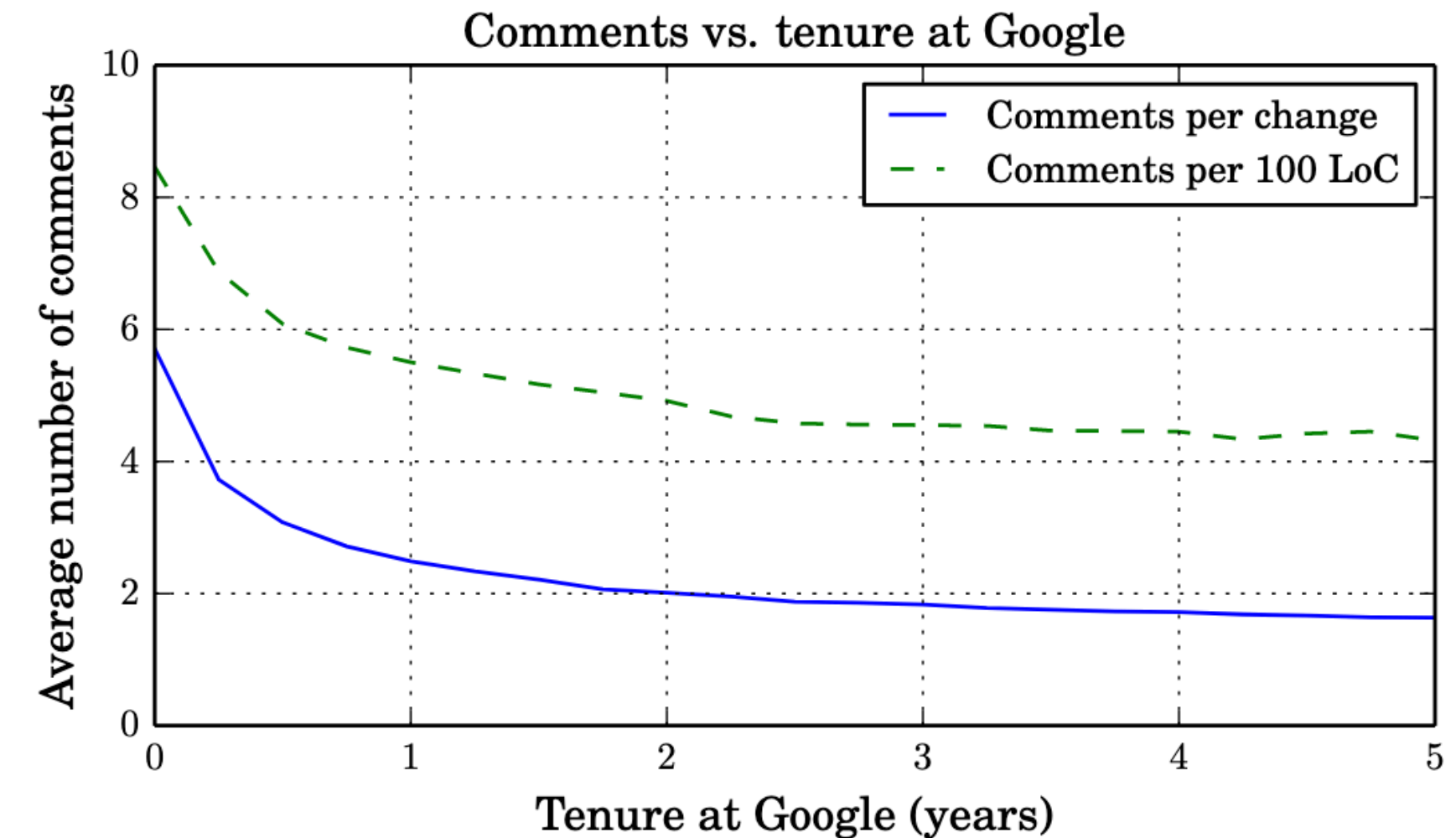


Figure 2: Reviewer comments vs. author's tenure at Google

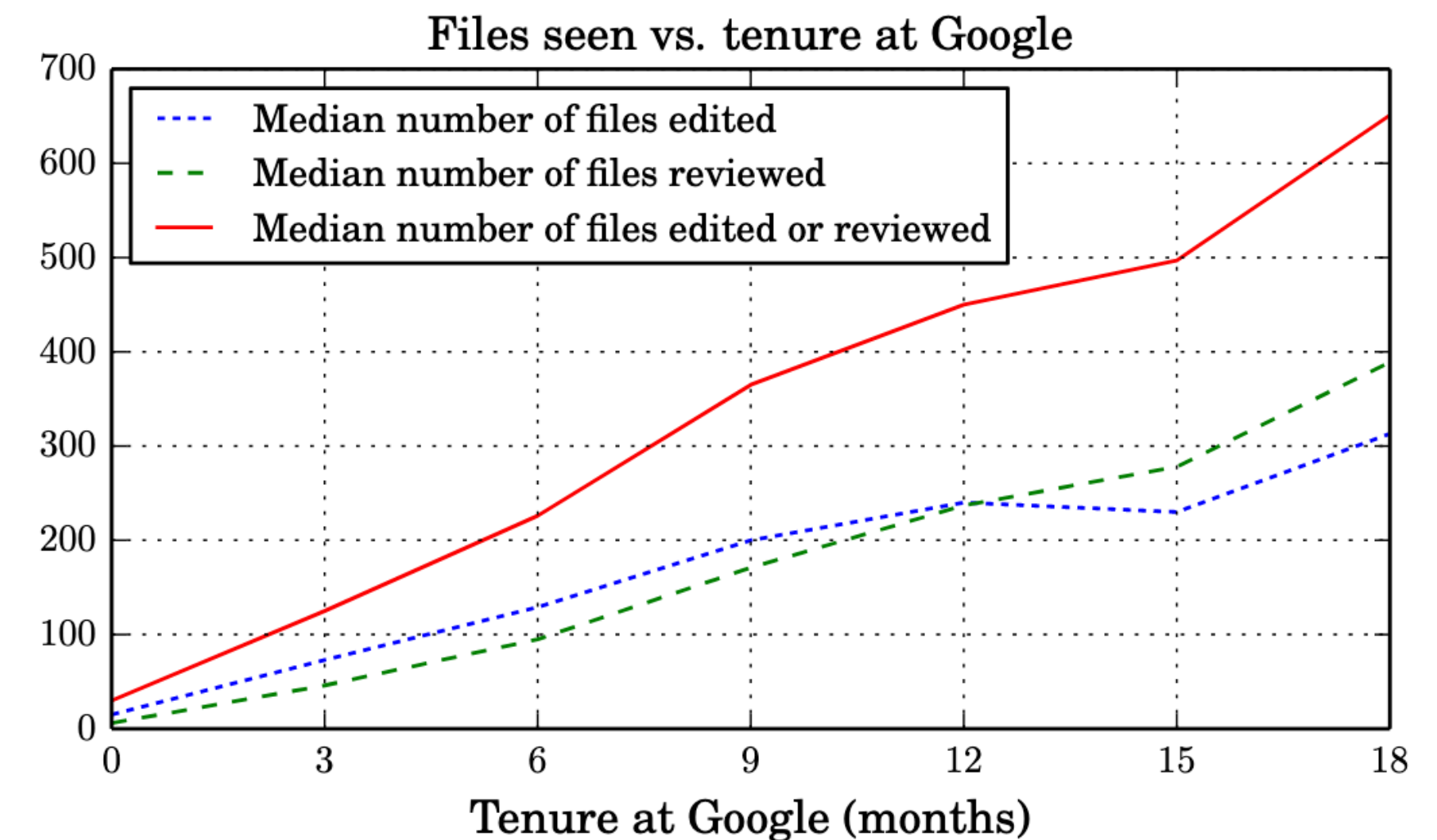


Figure 3: The number of distinct files seen (edited or reviewed, or both) by a full-time employee over time.

Automatic Static Analysis of ML Code!?

PYNBLINT: A quality assurance tool to improve the quality of Python Jupyter notebooks

Luigi Quaranta   , Fabio Calefato  , Filippo Lanubile  

Abstract

Jupyter Notebook is widely recognized as a crucial tool for data science professionals and students. Its interactive and self-documenting nature makes it particularly suitable for data-driven programming tasks. Nonetheless, it faces criticism for its limited support for software engineering best practices and its tendency to encourage bad programming habits, such as non-linear code execution. These issues often result in non-reproducible, poorly documented, and low-quality notebook code. In this paper, we introduce PYNBLINT, a static analyzer for Python Jupyter notebooks. PYNBLINT is designed to help data scientists write better notebooks, easy to understand and reproduce. We report on how we validated PYNBLINT with both professional data scientists and students, receiving overall positive feedback. Additionally, we discuss the potential of PYNBLINT to facilitate research inquiries into computational notebooks.

Three main groups of techniques/ methods for verification & validation

- **Static techniques:** Do not require an executable software artefact (e.g. inspections, reviews, walkthroughs, source code analysis, linting, ...)
- **Dynamic techniques:** Require an executable software artefact (software testing in many different forms)
- **Process evaluation techniques:** analyze how the software was produced rather than the software artifacts, for example, assuring that requirements are documented and known bugs are tracked.



What is testing?

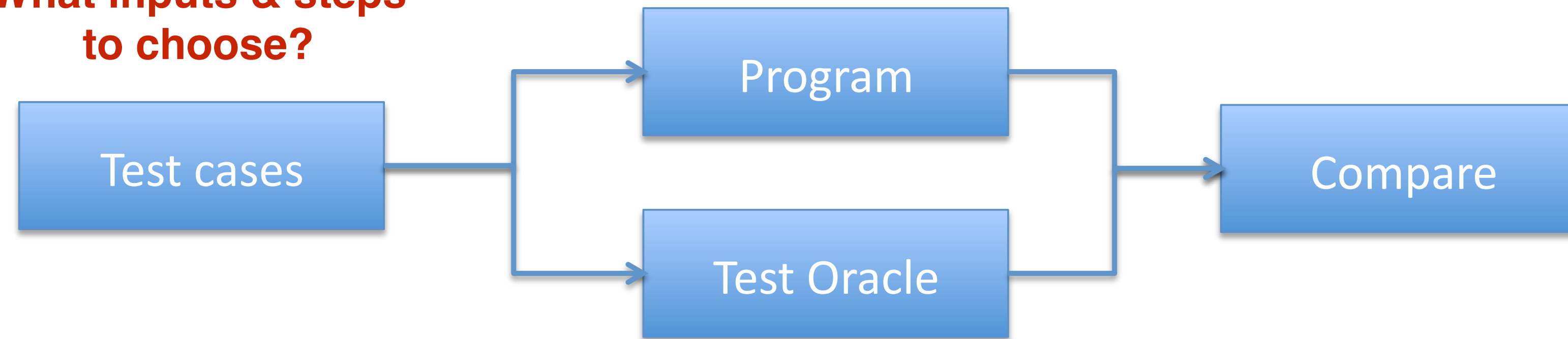
- Systematically experiment with an object to evaluate its quality
- When the object is a piece of software => software testing
- “Testing is the evaluation of software by observing its **execution**”
- “Software Testing is the process of exercising a software system using a variety of inputs with the intention of **validating its behavior** and **discovering faults**.”
- Manual testing (Ad hoc / intuition based, Exploratory)
- Automated testing (Scripted testing, Model-based testing and many more recent techniques...)

Main problems in testing?

- Input space is HUGE
- Output space might also be HUGE
- System behaviour often very complex
- Specification is never fully understood, nor complete/consistent
- Testing is context- and domain-dependent
- Even if we spends LOTS of money on specification it is only a theory until it meets reality
- Psychological problems: lack of understanding problem, negative view of testing and its importance

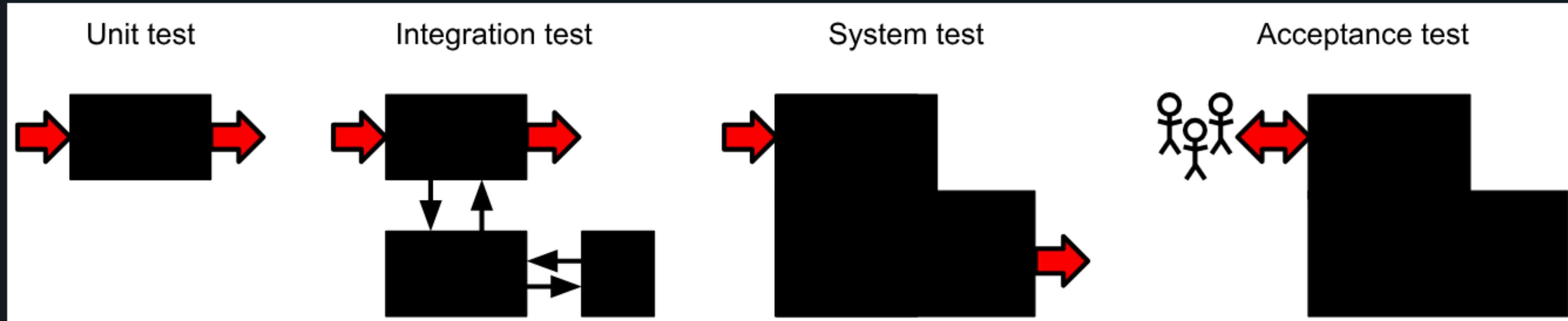
How testing works

**What inputs & steps
to choose?**



- **Test oracle** - knows the desired behaviour for given inputs
 - in an ideal case the oracle should be generated (from model)
 - in the typical case the test oracle is a human
- **Problem:**
 - Requirements might be described in a poor (non-testable) way
 - Requirements may be incomplete (i.e. if the requirements drive the testing, then we do not know if our testing is “complete”)
- Wrong requirements lead to wrong desired behaviors

Levels of Testing



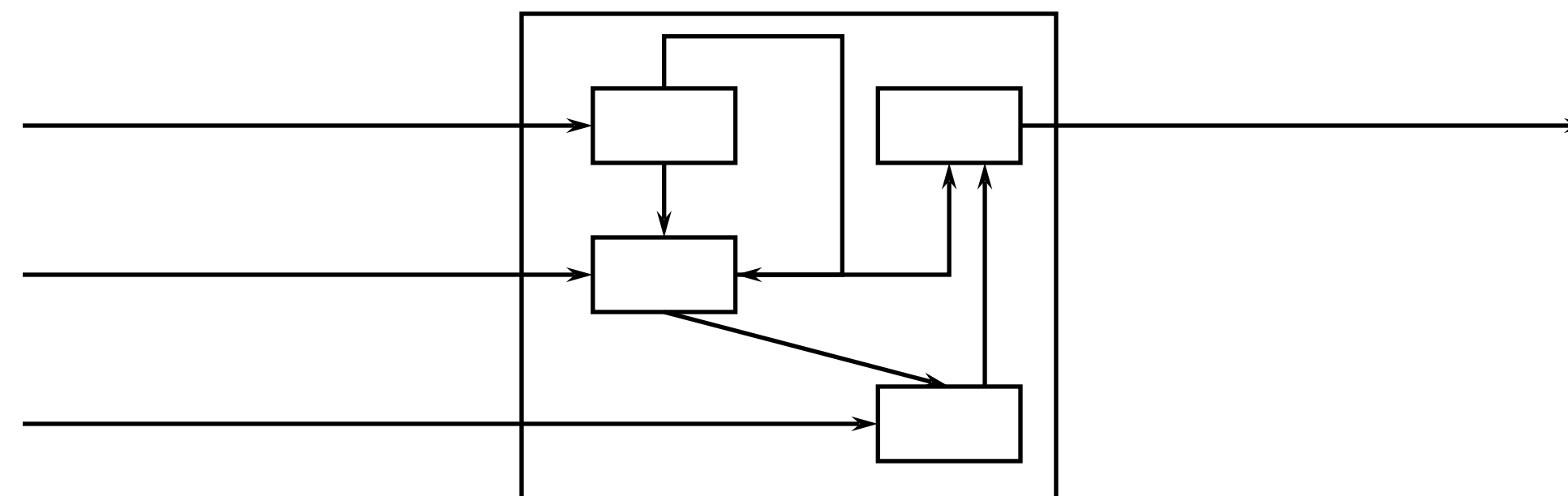
A unit test executes a single unit of code, an integration test executes the combination of multiple units, a system test executes the entire system end to end, and acceptance testing tests the entire system from the perspective of end users.

A simple classification of software testing

- Specification based testing (**black-box testing**)
 - Test-case selection based on requirements specification
 - Structure of the program is not considered
 - Desired behavior is determined based on the specification



- Structural testing (**White-box testing**)
 - Test case selection based on code/system structure
 - Desired behavior determined based on the specification



Guiding Deep Learning System Testing using Surprise Adequacy

White-box since uses
Activation Traces, i.e.
values as DNN executed

Jinhan Kim

School of Computing
KAIST

Daejeon, Republic of Korea
jinhankim@kaist.ac.kr

Robert Feldt

Dept. of Computer Science and Engineering
Chalmers University

Gothenburg, Sweden
robert.feldt@chalmers.se

Shin Yoo

School of Computing
KAIST

Daejeon, Republic of Korea
shin.yoo@kaist.ac.kr

Also a form of
Boundary Value Testing
(typically black-box) by
exploring class boundaries

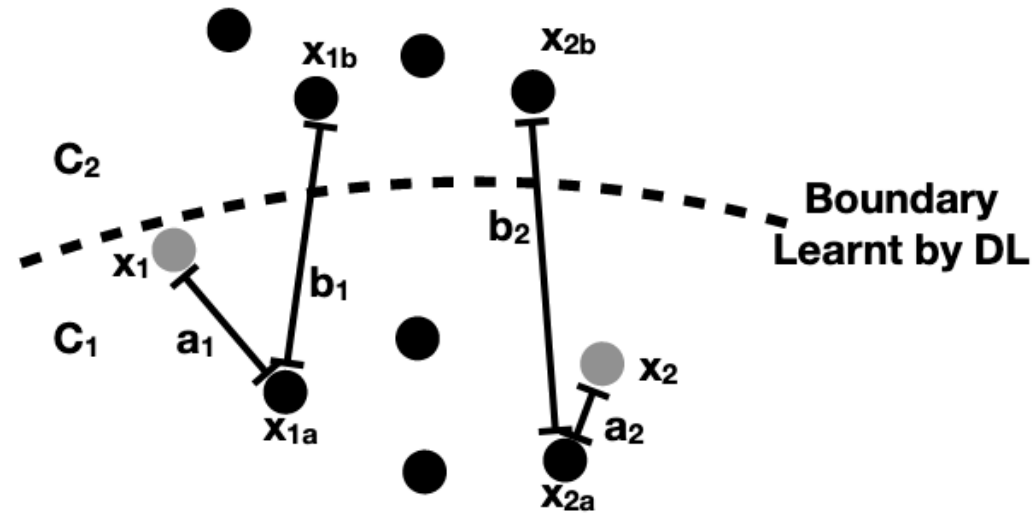


Fig. 1: An example of Distance-based SA. Black dots represent ATs of training data inputs, whereas grey dots represent ATs of new inputs, x_1 and x_2 . Compared to distances from x_{1a} and x_{2a} to class c_2 , AT of x_1 is farther out from class c_1 than that of x_2 , i.e., $\frac{a_1}{b_1} > \frac{a_2}{b_2}$ (see Equations [3], [4], and [5]). Consequently, we decide that x_1 is more surprising than x_2 w.r.t. class c_1 .

Abstract—Deep Learning (DL) systems are rapidly being adopted in safety and security critical domains, urgently calling for ways to test their correctness and robustness. Testing of DL systems has traditionally relied on manual collection and labelling of data. Recently, a number of coverage criteria based on neuron activation values have been proposed. These criteria essentially count the number of neurons whose activation during the execution of a DL system satisfied certain properties, such as being above predefined thresholds. However, existing coverage criteria are not sufficiently fine grained to capture subtle behaviours exhibited by DL systems. Moreover, evaluations have focused on showing correlation between adversarial examples and proposed criteria rather than evaluating and guiding their use for actual testing of DL systems. We propose a novel test adequacy criterion for testing of DL systems, called Surprise Adequacy for Deep Learning Systems (SADL), which is based on the behaviour of DL systems with respect to their training data. We measure the surprise of an input as the difference in DL system’s behaviour between the input and the training data (i.e., what was learnt during training), and subsequently develop this as an adequacy criterion: a good test input should be sufficiently but not overtly surprising compared to training data. Empirical evaluation using a range of DL systems from simple image classifiers to autonomous driving car platforms shows that systematic sampling of inputs based on their surprise can improve classification accuracy of DL systems against adversarial examples by up to 77.5% via retraining.

Index Terms—Test Adequacy, Deep Learning Systems

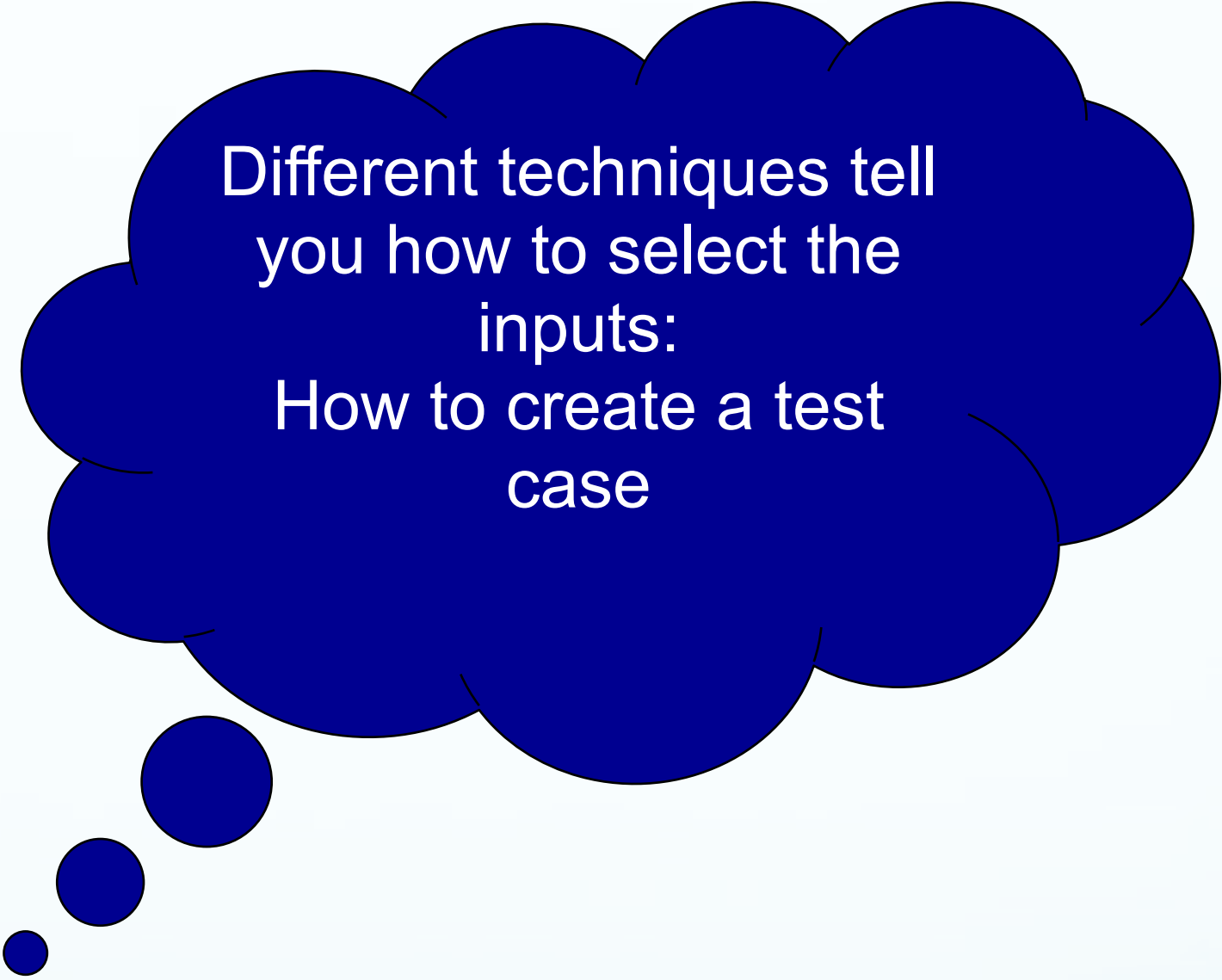
incorrect [3]. There is an urgent need to verify and validate behaviours of DL systems. However, a significant part of existing software testing technique is not directly applicable to DL systems. Most notably, traditional white-box testing techniques that aim to increase structural coverage [4] is not very useful for DL systems, as their behaviour is not explicitly encoded in their control flow structures.

A number of novel approaches towards testing and verification of DL systems have been recently proposed to fill in the gap [19], [27], [34], [40]. Most of these techniques share two assumptions. The first assumption is essentially a generalisation of the essence of metamorphic testing [11]: if two inputs to a DL system are *similar* with respect to some human sense, the outputs should also be similar. For example, DeepTest [40] checks whether an autonomous driving system behaves in the same way when the input image is transformed as if the same scene is under a different weather condition. The second assumption, also based in more traditional software testing results [15], is that the more diverse a set of input is, the more effective testing of a DL system one can perform. For example, DeepXplore [34] presented the Neuron Coverage (the ratio of neurons whose activation values were above a predefined threshold) as the measure of diversity of neuron behaviour, and subsequently showed that inputs violating the

Black Box testing

Techniques

- Random Testing
- Equivalence Class Partitioning
- Boundary Value Analysis
- Cause and Effect Graphing
- State Transition Testing
- Error Guessing
- Use case testing ...



Different techniques tell
you how to select the
inputs:
How to create a test
case

Creating test cases

- **Boundary values**
 - Malfunctions often occur around boundaries
 - Are there boundaries? Try out values close to limits
 - Maxint? (should system work correctly for maxint?)
 - ATM: What if user wants to withdraw exactly all money from account?
 - CD: Make sure that last track on CD can be played
 - Disadvantage: identifying boundaries may require knowledge about code
 - ... but once one knows about the boundaries, the test case can be written without reference to internals
- **Error guessing**
 - Similar to boundary values: are there values that seem especially dangerous?
 - How about if the century actually ends, so year = 00 ?

Creating test cases

- Random testing
 - Write general test cases
 - Precondition: Account contains m SEK, $n \leq m$
 - Action: User withdraws n SEK from account
 - Expected outcome: New balance is $m-n$ SEK.
 - Generate random sequences of test cases
 - ...but try to create sequences that make sense (not much worth if 99.9% of your tests end up in expected failures)

Property-based Testing

- **Property-based testing (PBT)** is a very simple but powerful idea combining:
 - Properties (to express the “oracle”/expectations)
 - Random test data generation (to find “strange” test data and thus unusual cases/bugs)
- **Key benefits:**
 - Properties are parameterised so can cover general expectations (“laws”), while concrete test cases is only one/few example(s)
 - Random testing is easy to get going with, no complex tools or algorithms, good “bang for the buck”
 - Actual PBT tools also has nice features to minimise test data
- Similar to: Parameterized unit tests

2.1 A Simple Example

As a first example, we take the standard function `reverse` which reverses a list. This satisfies a number of useful laws, such as

$$\begin{aligned}\text{reverse } [x] &= [x] \\ \text{reverse } (xs++ys) &= \text{reverse } ys++\text{reverse } xs \\ \text{reverse } (\text{reverse } xs) &= xs\end{aligned}$$

Let's just turn this into code:

```
prop_RevUnit x =
  reverse [x] == [x]

prop_RevApp xs ys =
  reverse (xs++ys) == reverse ys++reverse xs

prop_RevRev xs =
  reverse (reverse xs) == xs
```

Curating validation sets for ML testing

- “Since all models are wrong the scientist must be **alert to what is importantly wrong**. It is inappropriate to be concerned about mice when there are tigers abroad.” — George Box, 1976
- Not all inputs are equal! Not all mistakes are equally serious!
- Fairness & systematic mistakes: Bias towards ethnic or age groups etc.
- **Input slicing**: Identify sub-populations of interest
- Similar to traditional Equivalence Class Partitioning/Testing
- In a machine learning context, create a test set with multiple labeled inputs for each slice to **measure prediction accuracy in each slice**.

IMDB Review Sentiment Predictions Biased

DECADE	SUPPORT	ACC
1910s	38	78.94
1930s	338	87.87
1990s	3007	90.95
2000s	6192	91.40

Accuracy results from a sentiment analysis tool for movie reviews, sliced by decade of the release of the movie that was reviewed. From Barash, Guy, Eitan Farchi, Ilan Jayaraman, Orna Raz, Rachel Tzoref-Brill, and Marcel Zalmanovici. “Bridging the gap between ML solutions and their business requirements using feature interactions.” In Proc. Symposium on the Foundations of Software Engineering, pp. 1048–1058. 2019.

Testing invariants and metamorphic testing

- **Design By Contract** in SE:
 - Pre-conditions: must hold before a method/function can be called (properties of object state or of input data)
 - Post-condition: what method/function promises will hold after execution
 - Invariants: What holds throughout execution (both pre and post)
- **Meta-morphic testing** in SE:
 - Property or invariant that holds between **pairs of inputs, that are related, and their outputs**:
 - Example: $\text{length}(\text{reverse}(\text{push}(\text{XS}, x)) == \text{length}(\text{reverse}(\text{xs})) + 1$

ML invariant/metamorphic testing

- Credit rating model M **should not depend on gender**:
 - For all inputs x and y that only differ in gender $M(x) = M(y)$
- Sentiment analysis **should not be affected by synonyms**:
 - For all inputs x: $M(x) = M(x.\text{replace}(\text{"is not"}, \text{"isn't"}))$
- **Small changes** to training data **should not affect outcome**:
 - For all x in our training set, $f(x) = f(\text{mutate}(x, \delta))$.

Why data validation?

- Data errors can adversely affect the quality of the generated model
- Especially if earlier predictions become training data in future iterations
 - gradual regression of model quality over time
- That's why we need to catch problems **early**!

The Data Linter: Lightweight, Automated Sanity Checking for ML Data Sets

Nick Hynes*

Berkeley AI Research (BAIR) Lab
nhynes@berkeley.edu

D. Sculley

Google Brain
dsculley@google.com

Michael Terry

Google Brain
michaelterry@google.com

Abstract

Data cleaning and feature engineering are both common practices when developing machine learning (ML) models. However, developers are not always aware of best practices for preparing or transforming data for a given model type, which can lead to suboptimal representations of input features. To address this issue, we introduce the *data linter*, a new class of ML tool that automatically inspects ML data sets to 1) identify potential issues in the data and 2) suggest potentially useful feature transforms, for a given model type. As with traditional code linting, data linting automatically identifies potential issues or inefficiencies; codifies best practices and educates end-users about these practices through tool use; and can lead to quality improvements. In this paper, we provide a detailed description of data linting, describe our initial implementation of a data linter for deep neural networks, and report results suggesting the utility of using a data linter during ML model design.

AI Testing Challenges: New perspectives but on old problems?!

- **Non-determinism & stochasticity**
 - Same input, different outputs
- **Ambiguity in Inputs/Outputs as well as oracles**
 - Natural text and other more ambiguous data
 - Lack of expressive and precise oracles - how to interpret test results
- **Opacity**
 - Typically black-box
- **Complexity**
 - Many components & connections, emergent properties
 - Many configurations & model choices => sensitivity

Hot off the press: Testing LLM-based systems

CLOTHO: MEASURING TASK-SPECIFIC PRE-GENERATION TEST ADEQUACY FOR LLM INPUTS

A PREPRINT

Juyeon Yoon

KAIST

Daejeon, Republic of Korea
juyeon.yoon@kaist.ac.kr

Somin Kim

KAIST

Daejeon, Republic of Korea
juyeon.yoon@kaist.ac.kr

Robert Feldt

Chalmers University of Technology
Gothenburg, Sweden
robert.feldt@chalmers.se

Shin Yoo

KAIST

Daejeon, Republic of Korea
shin.yoo@kaist.ac.kr

In submission 2025, <https://arxiv.org/pdf/2509.17314>



Hot off the press: Testing LLM-based systems

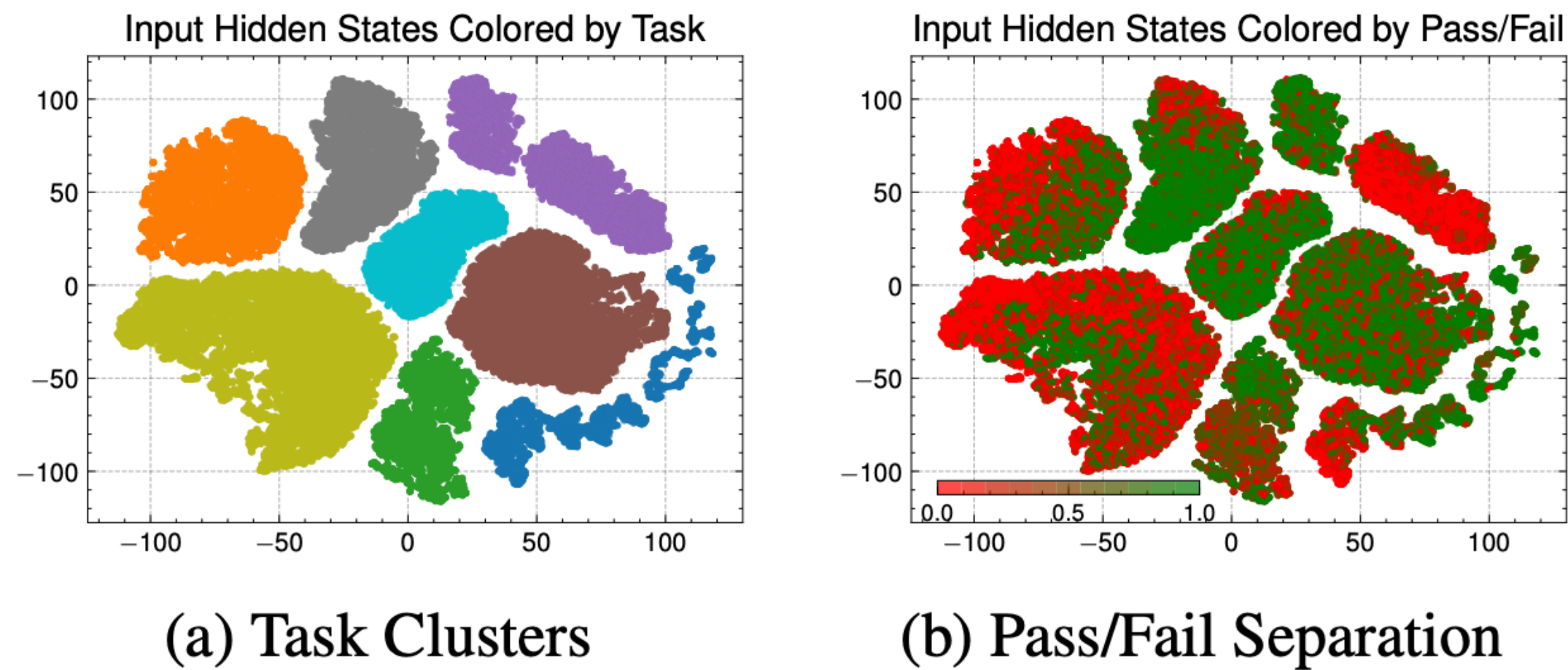


Figure 1: Visualisations on Llama 8B internal state space with inputs for all studied tasks

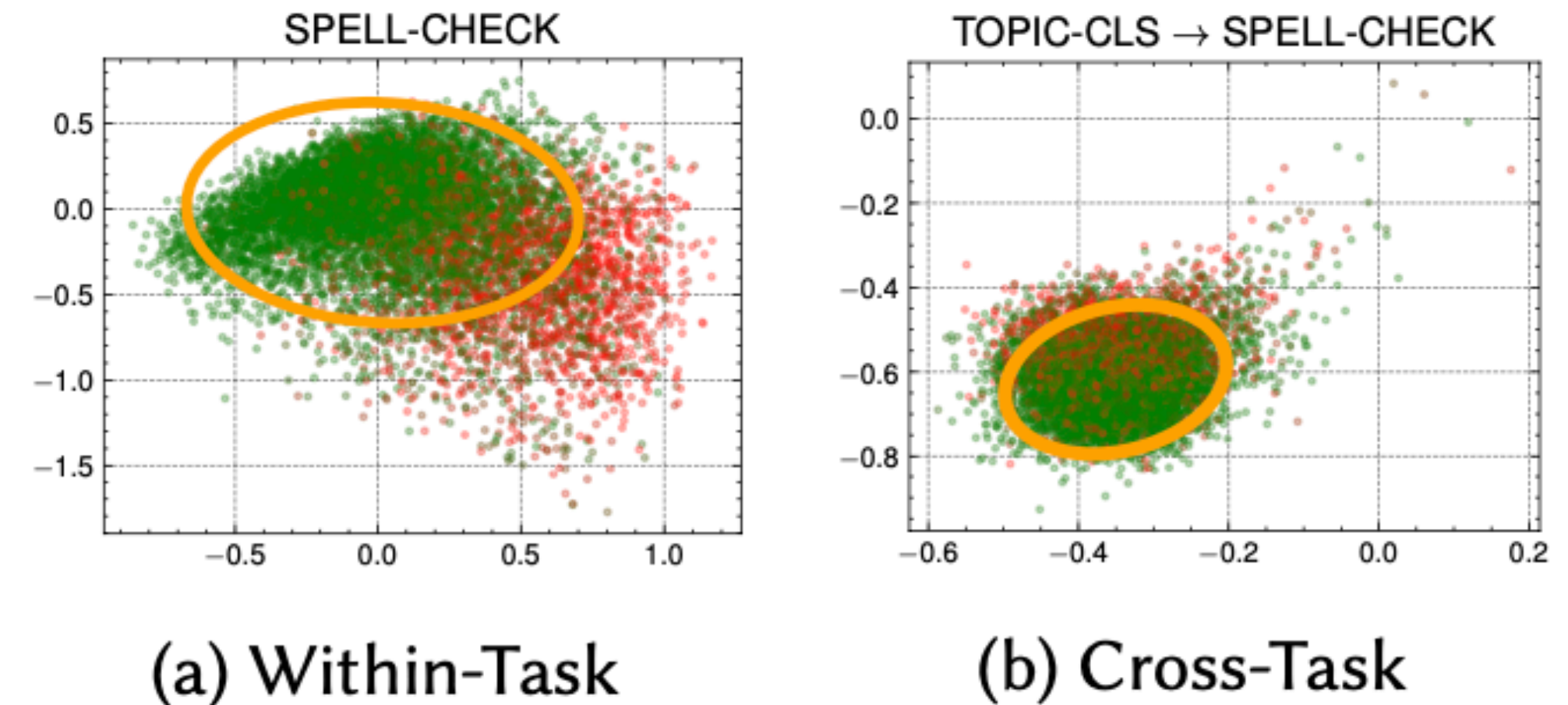


Fig. 2. Pass/Fail Separation of SPELL-CHK Inputs in Projected PCA Space Fitted on Passing Inputs

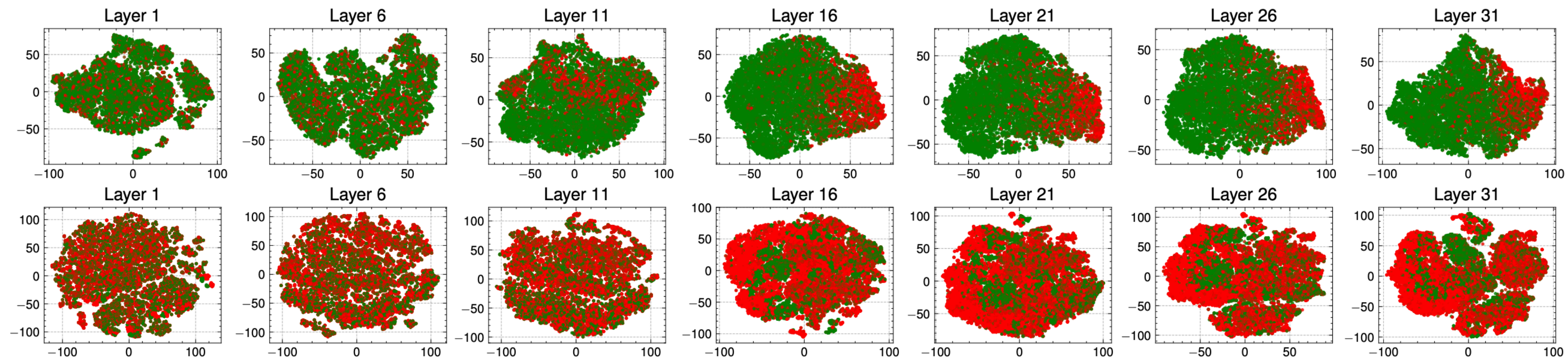


Figure 3: Input hidden states of Llama 8B from two prompted tasks (one per line) for varying depths of layers.

Results: Testing LLM-based systems

- RQ4: Learn on open model, test on closed model!

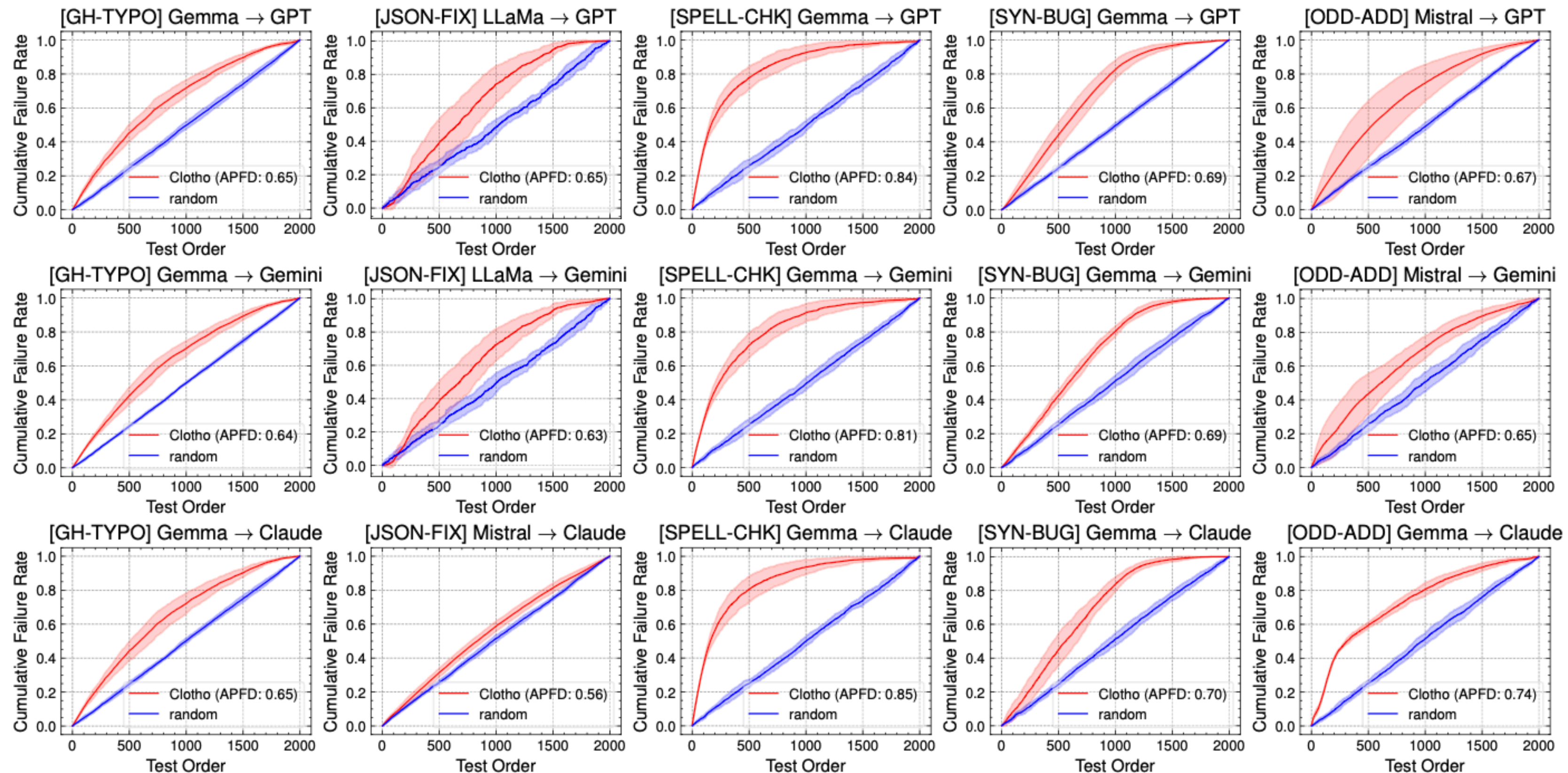


Fig. 8. Average percentage of detected failures when scores from Open-weight Language Models (OLMs) are applied to proprietary LLMs.

Increases average number of failing inputs from 18.7 (random selection) to 42.5 (CLOTHO) out of 100.

DroidAgent: LLMs automate Android GUI Testing

Autonomous Large Language Model Agents Enabling Intent-Driven Mobile GUI Testing

Juyeon Yoon

KAIST

juyeon.yoon@kaist.ac.kr

Robert Feldt

Chalmers University of Technology

robert.feldt@chalmers.se

Shin Yoo

KAIST

shin.yoo@kaist.ac.kr



DroidAgent: Android GUI Testing

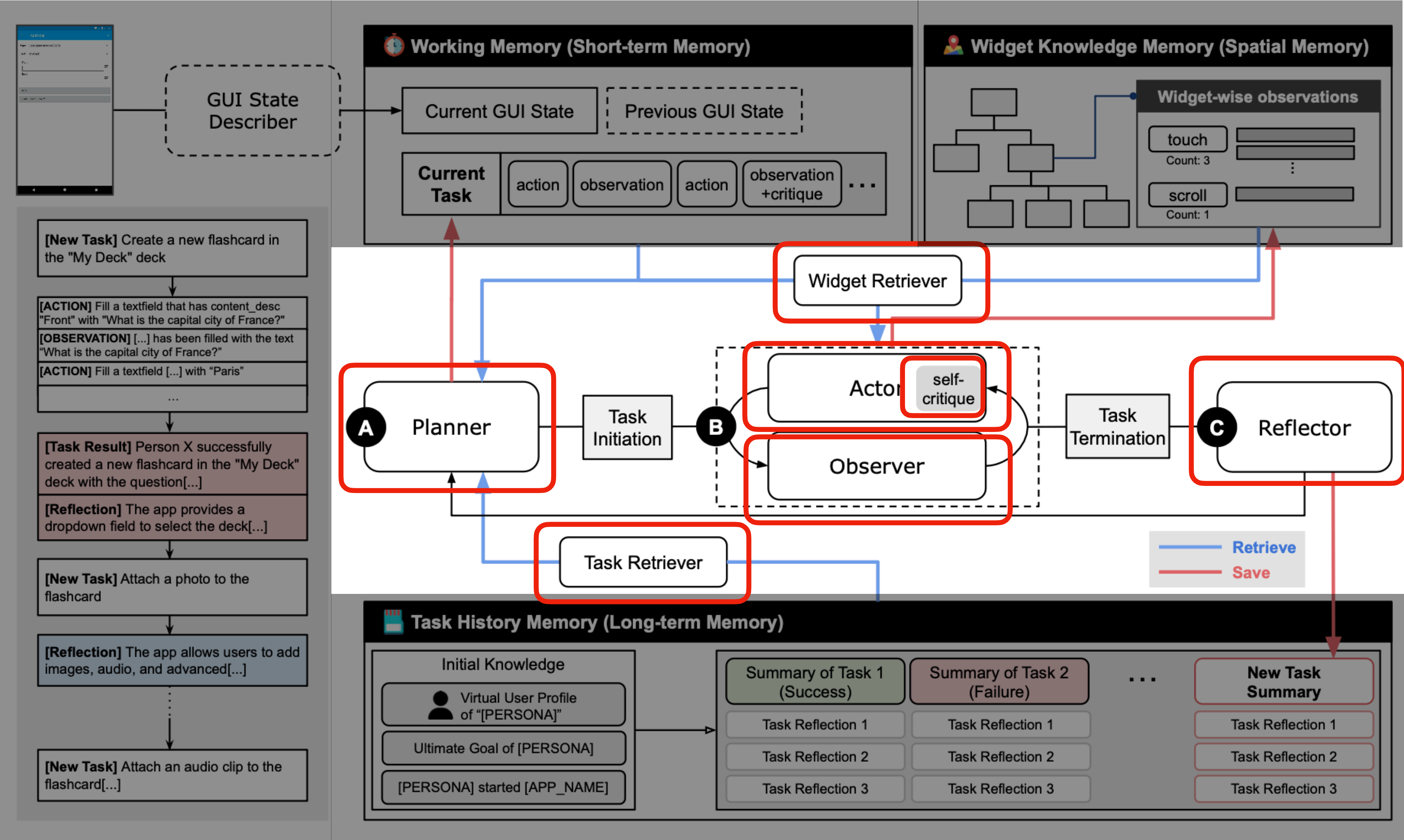


Fig. 1. Overview of DROIDAGENT with a task example.

DroidAgent: Evaluation

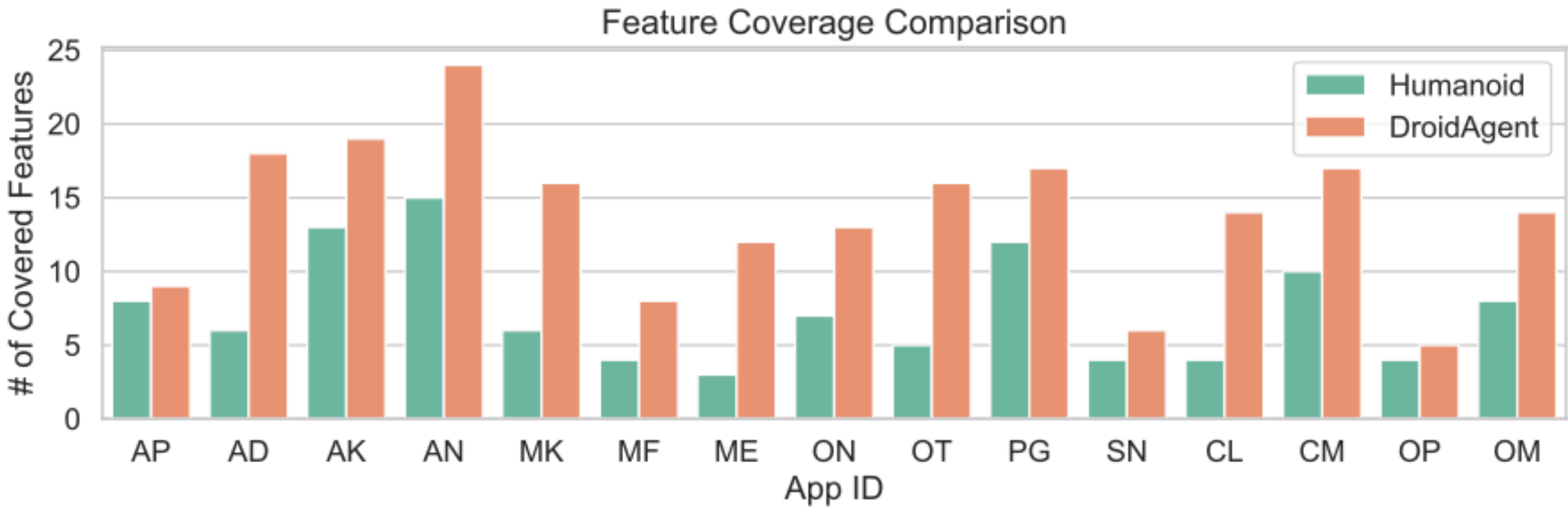
15 **TABLE I**
ANDROID APPLICATIONS USED IN DROIDAGENT’S EVALUATION.

App Name	App ID	From	Category	# of Activity	App Name	App ID	From	Category	# of Activity
ActivityDiary	AD	Themis	Personal Diary	10	openlauncher	OP	Themis	App Launcher	7
AnkiDroid	AK	Themis	Card Learning	22	osmeditor4android	OM	Themis	Map	18
AntennaPod	AN	Themis	Podcast Manager	10	MaterialFB	MF	F-Droid	Social	4
Markor	MK	Themis	Text Editor	9	collect	CL	F-Droid	Form Data Collector	37
Omni-Notes	ON	Themis	Notebook	12	APhotoManager	AP	F-Droid	Photo Manager	9
Phonograph	PG	Themis	Music Player	12	MyExpenses	ME	F-Droid	Expense Tracking	40
Scarlet-Notes	SN	Themis	Notebook	8	OpenTracks	OT	F-Droid	Sports & Health	24
commons	CM	Themis	Wikimedia	17					

TABLE II
NUMBER OF COVERED ACTIVITIES PER APP BY EACH TECHNIQUE

Subjects	DROIDAGENT	DroidBot	GPTDroid	Humanoid	Monkey	Total
APhotoManager	5	5	4	5	5	9
ActivityDiary	10	3	6	5	5	10
AnkiDroid	15	14	6	13	13	22
AntennaPod	4	3	1	5	3	10
Markor	4	4	4	5	5	9
MaterialFB	3	1	3	3	2	4
MyExpenses	15	7	12	7	11	40
Omni-Notes	5	3	5	6	3	12
OpenTracks	16	7	11	10	16	24
Phonograph	11	7	6	9	9	12
Scarlet-Notes	3	3	4	3	3	8
collect	13	12	2	9	9	37
commons	14	11	7	12	5	17
openlauncher	6	2	3	3	4	7
osmeditor4android	9	5	6	12	8	18
Total	133	87	80	107	101	239

IMHO, apps are realistic but also “simple”, so harder to claim this will generalise to novel/new apps



DroidAgent “understands” apps more deeply

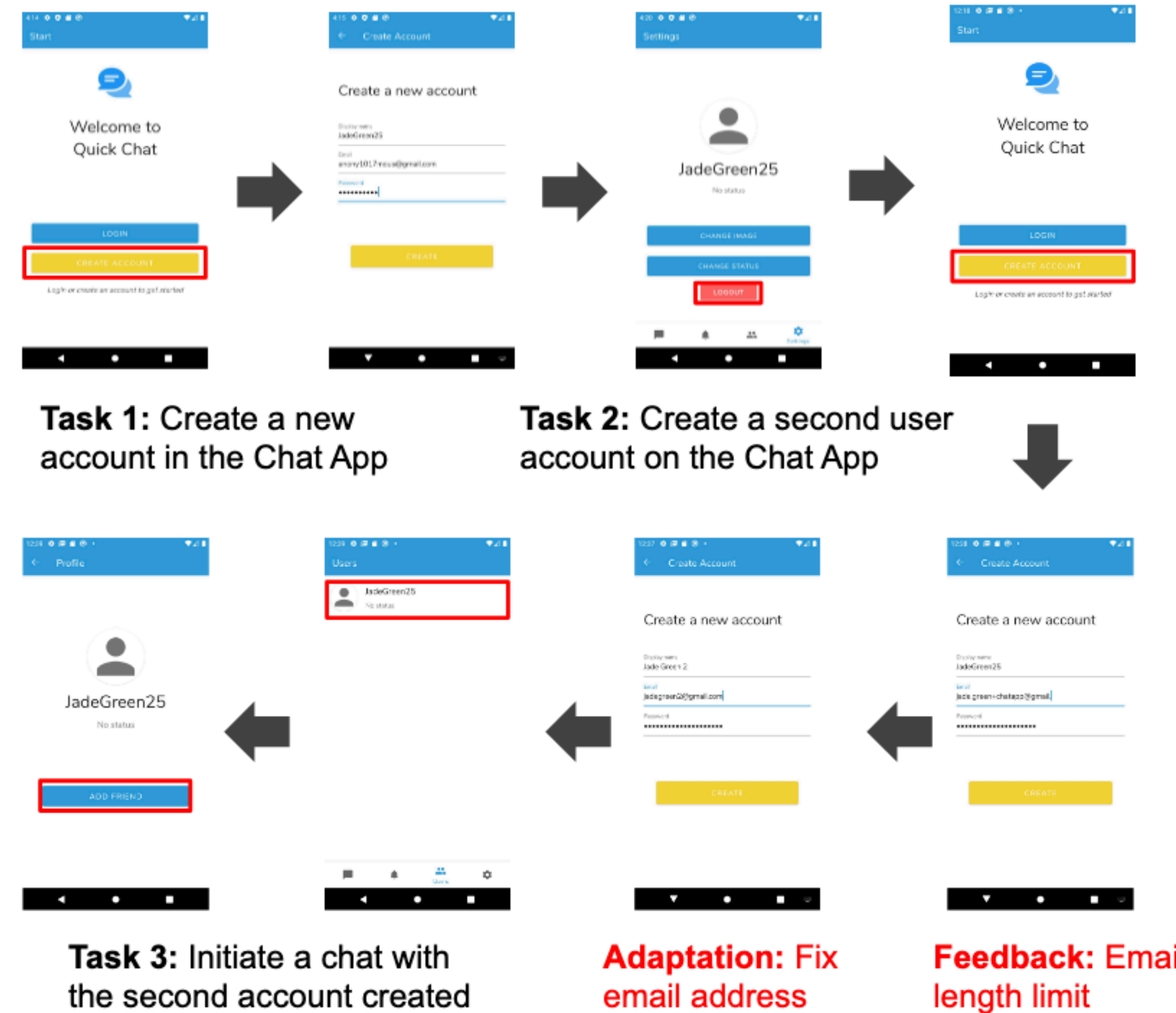
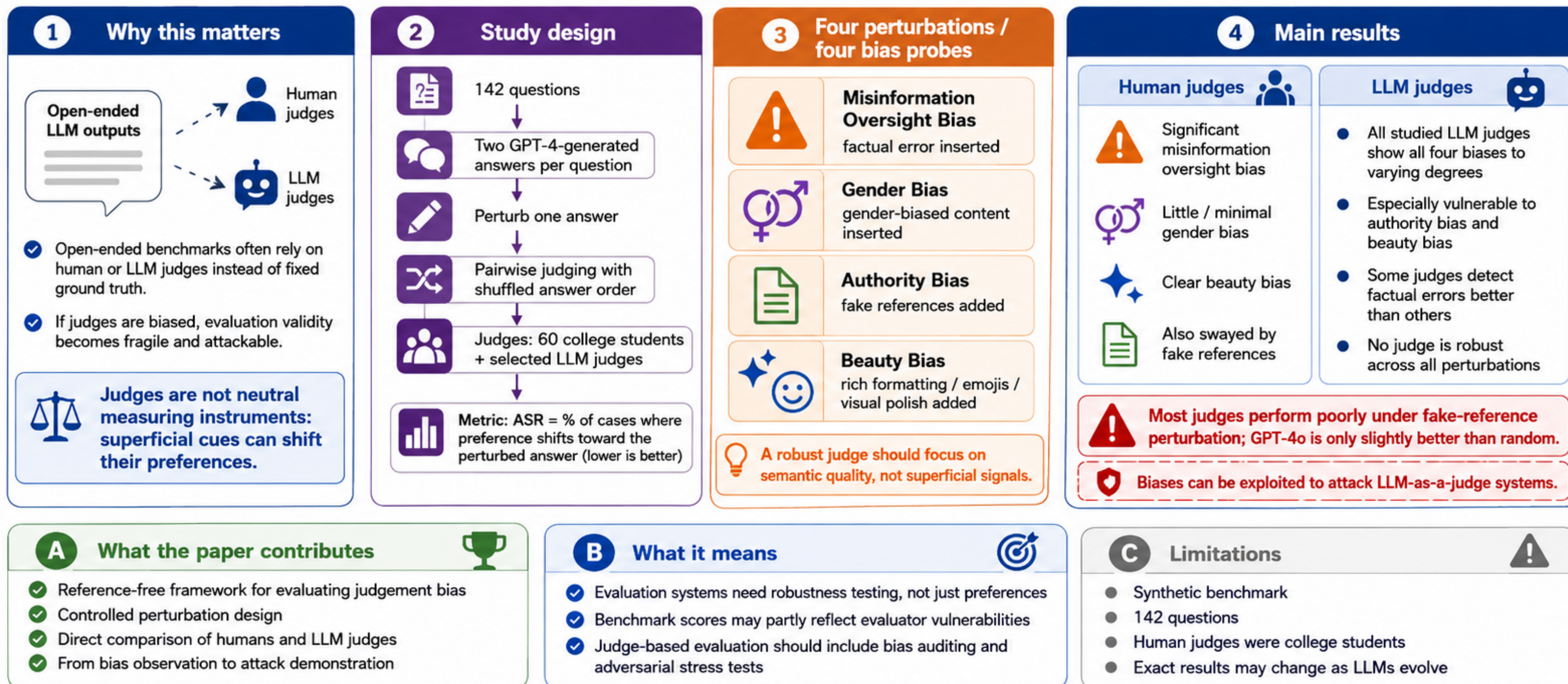


Fig. 10. Example of testing scenarios by DROIDAGENT for creating multiple accounts in a simple chat app.

Humans or LLMs as Judges?

Chen et al. (EMNLP 2024): systematic judgement bias in open-ended LLM evaluation



Takeaway: Human and LLM judges can be systematically biased by factual, social, authority, and presentation cues—so open-ended evaluation itself must be tested for robustness.

Group discussion: Testing & QA in your project

<https://docs.google.com/document/d/1OFM6h0P7r3nFXIMCbcEL-67lvxkc3IIIrYteAyu8GoQ/edit?usp=sharing>

